

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería
Escuela de Ciencias y Sistemas

Proyecto GoLite IDE

Manual Técnico

Fase 1

Curso: Organización de Lenguajes y Compiladores 1

Estudiante:

Dalio Antonio Miranda Guerra

Carné:

202100116

Guatemala, Junio de 2026

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Tecnologías Utilizadas	3
3.1. Java	3
3.2. Maven	3
3.3. JFlex	3
3.4. CUP	4
3.5. Swing	4
4. Arquitectura del Proyecto	4
5. Análisis Léxico	5
6. Análisis Sintáctico	6
7. Árbol de Sintaxis Abstracta	7
8. Módulo Intérprete	8
9. Reportes Generados	9
9.1. Reporte de Tokens	9
9.2. Reporte de Errores	9
10. Compilación del Proyecto	10
11. Ejecución del Sistema	10
12. Conclusiones	11

Índice de figuras

1.	Estructura general del proyecto GoLite	4
2.	Archivo golite.flex utilizado para el análisis léxico	5
3.	Archivo parser.cup utilizado para el análisis sintáctico	6
4.	Clases utilizadas para representar nodos del AST	7
5.	Clase Interpreter.java	8
6.	Reporte de tokens generado por el IDE	9
7.	Compilación exitosa del proyecto con Maven	10
8.	Ejecución correcta de un programa GoLite	11

1. Introducción

El presente manual técnico describe la estructura interna, tecnologías utilizadas y funcionamiento general del proyecto GoLite IDE, desarrollado como parte de la Fase 1 del curso Organización de Lenguajes y Compiladores 1.

El sistema consiste en un intérprete para un subconjunto del lenguaje GoLite. Para su construcción se utilizaron herramientas propias del área de compiladores, tales como JFlex para el análisis léxico y CUP para el análisis sintáctico. Además, se implementó una interfaz gráfica en Java para permitir la escritura, edición, ejecución y visualización de resultados del código fuente.

2. Objetivos

2.1. Objetivo General

Desarrollar un intérprete funcional para el lenguaje GoLite mediante Java, JFlex y CUP, integrando una interfaz gráfica que permita la ejecución de programas y la generación de reportes.

2.2. Objetivos Específicos

- Implementar un analizador léxico capaz de reconocer tokens del lenguaje GoLite.
- Implementar un analizador sintáctico que valide la estructura gramatical del código fuente.
- Ejecutar instrucciones básicas del lenguaje mediante una estructura basada en nodos.
- Mostrar resultados y errores mediante una consola integrada en la interfaz gráfica.
- Generar reportes de tokens y errores para facilitar el análisis del código.

3. Tecnologías Utilizadas

3.1. Java

Java fue utilizado como lenguaje principal para implementar la lógica del sistema, la interfaz gráfica, los nodos del intérprete y el manejo de reportes.

3.2. Maven

Maven fue utilizado para administrar dependencias, compilar el proyecto y generar el archivo ejecutable JAR.

3.3. JFlex

JFlex fue utilizado para construir el analizador léxico. Este archivo se encarga de reconocer palabras reservadas, operadores, identificadores, números, cadenas, comentarios y demás símbolos del lenguaje GoLite.

3.4. CUP

CUP fue utilizado para construir el analizador sintáctico. A partir de reglas gramaticales, el parser valida la estructura del programa y genera los nodos necesarios para la ejecución.

3.5. Swing

Swing fue utilizado para desarrollar la interfaz gráfica del IDE, incluyendo el editor de código, consola de salida, menús y ventanas de reportes.

4. Arquitectura del Proyecto

La estructura del proyecto está organizada en carpetas que separan los módulos principales del sistema.

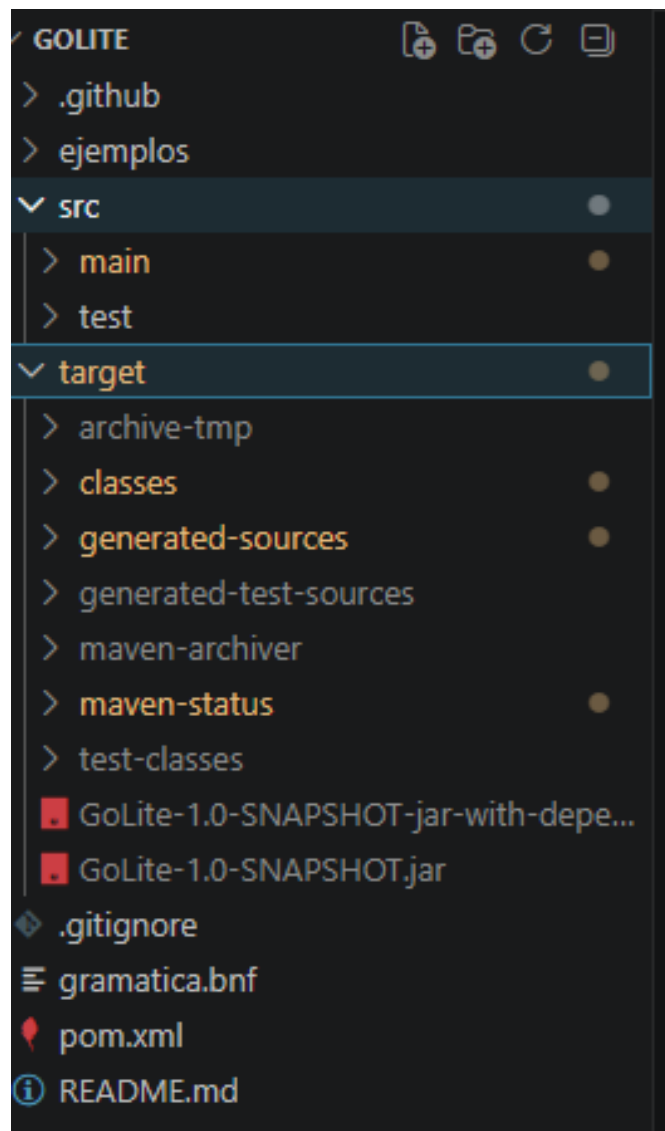
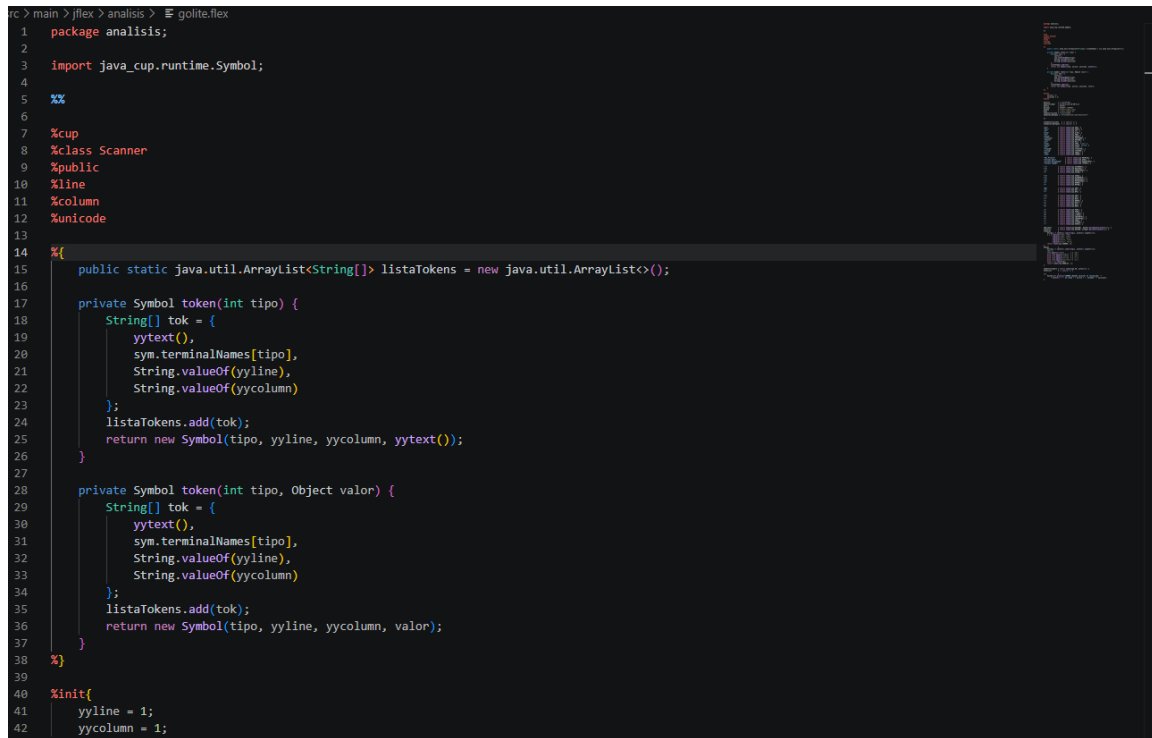


Figura 1: Estructura general del proyecto GoLite

La carpeta `src/main` contiene los archivos principales del proyecto. Dentro de ella se encuentran los archivos de CUP, JFlex y el código Java. La carpeta `ast` contiene los nodos utilizados para representar instrucciones y expresiones. La carpeta `gui` contiene la interfaz gráfica, mientras que la carpeta `interpreter` contiene la lógica de ejecución.

5. Análisis Léxico

El análisis léxico se realiza mediante el archivo `golite.flex`, ubicado dentro de la carpeta `src/main/jflex/analisis`.



```
1 package analisis;
2
3 import java_cup.runtime.Symbol;
4
5 %%
6
7 %cup
8 %class Scanner
9 %public
10 %line
11 %column
12 %unicode
13
14 %{
15     public static java.util.ArrayList<String[]> listaTokens = new java.util.ArrayList<>();
16
17     private Symbol token(int tipo) {
18         String[] tok = {
19             yytext(),
20             sym.terminalNames[tipo],
21             String.valueOf(yyline),
22             String.valueOf(yycolumn)
23         };
24         listaTokens.add(tok);
25         return new Symbol(tipo, yyline, yycolumn, yytext());
26     }
27
28     private Symbol token(int tipo, Object valor) {
29         String[] tok = {
30             yytext(),
31             sym.terminalNames[tipo],
32             String.valueOf(yyline),
33             String.valueOf(yycolumn)
34         };
35         listaTokens.add(tok);
36         return new Symbol(tipo, yyline, yycolumn, valor);
37     }
38 }%
39
40 %init{
41     yyline = 1;
42     yycolumn = 1;
```

Figura 2: Archivo `golite.flex` utilizado para el análisis léxico

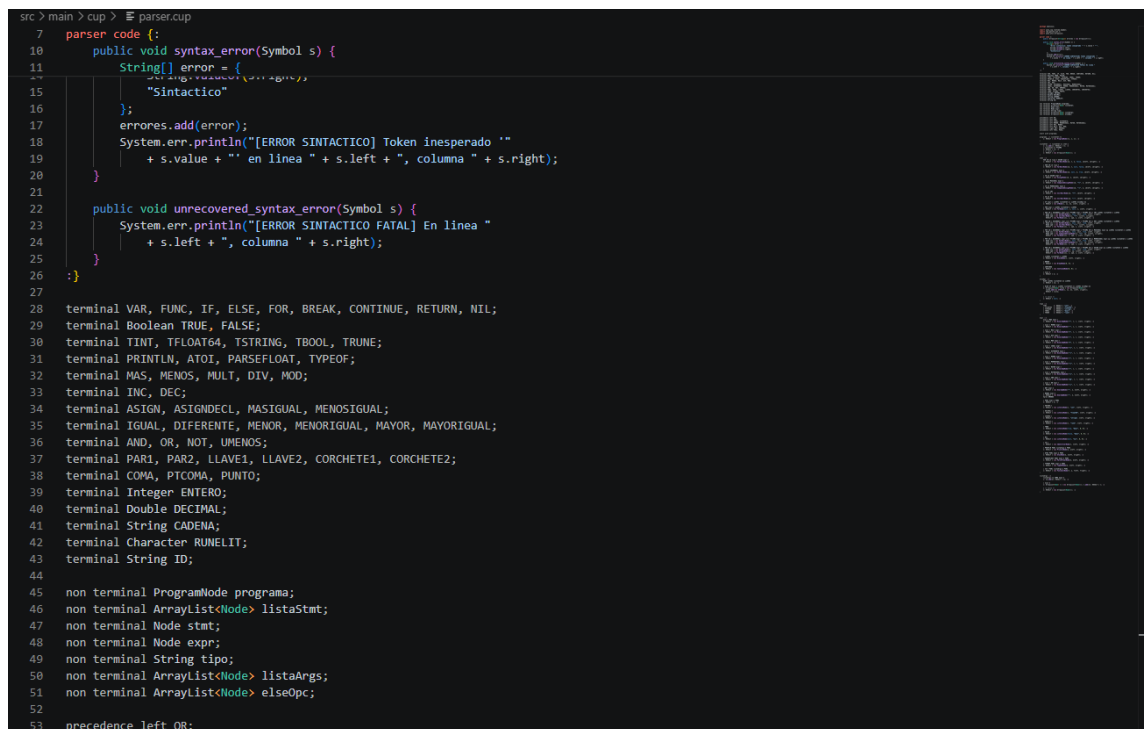
Este archivo define las expresiones regulares utilizadas para reconocer los componentes básicos del lenguaje. Entre los elementos reconocidos se encuentran:

- Palabras reservadas: `var`, `if`, `else`, `for`, `break`, `continue`.
- Tipos de datos: `int`, `float64`, `string`, `bool`, `rune`.
- Operadores aritméticos, relacionales y lógicos.
- Identificadores.
- Literales numéricos, cadenas y caracteres.
- Comentarios de una línea y comentarios multilínea.

Además, el archivo utiliza la configuración `%unicode`, lo cual permite reconocer caracteres especiales dentro del archivo fuente sin generar errores internos del scanner.

6. Análisis Sintáctico

El análisis sintáctico se realiza mediante el archivo `parser.cup`, ubicado dentro de la carpeta `src/main/cup`.



```
src > main > cup > parser.cup
7  parser code {
10  public void syntax_error(Symbol s) {
11      String[] error = {
12          "Sintactico"
13      };
14      errores.add(error);
15      System.err.println("[ERROR SINTACTICO] Token inesperado '"
16          + s.value + "' en línea " + s.left + ", columna " + s.right);
17  }
18
19  public void unrecovered_syntax_error(Symbol s) {
20      System.err.println("[ERROR SINTACTICO FATAL] En línea "
21          + s.left + ", columna " + s.right);
22  }
23
24  :}
25
26
27  terminal VAR, FUNC, IF, ELSE, FOR, BREAK, CONTINUE, RETURN, NIL;
28  terminal Boolean TRUE, FALSE;
29  terminal TINT, TFloat64, TString, TBool, TRune;
30  terminal TPrintln, TAtoi, TParsefloat, TTypeof;
31  terminal MAS, MENOS, MULT, DIV, MOD;
32  terminal INC, DEC;
33  terminal ASIGN, ASIGNDECL, MASIGUAL, MENOSIGUAL;
34  terminal IGUAL, DIFERENTE, MENOR, MENORIGUAL, MAYOR, MAYORIGUAL;
35  terminal AND, OR, NOT, UMENOS;
36  terminal PARI, PAR2, LLAVE1, LLAVE2, CORCHETE1, CORCHETE2;
37  terminal COMA, PTCOMA, PUNTO;
38  terminal Integer ENTERO;
39  terminal Double DECIMAL;
40  terminal String CADENA;
41  terminal Character RUNELIT;
42  terminal String ID;
43
44
45  non terminal ProgramNode programa;
46  non terminal ArrayList<Node> listaStmt;
47  non terminal Node stmt;
48  non terminal Node expr;
49  non terminal String tipo;
50  non terminal ArrayList<Node> listaArgs;
51  non terminal ArrayList<Node> elseOpc;
52
53  precedence left OR;
```

Figura 3: Archivo `parser.cup` utilizado para el análisis sintáctico

Este archivo contiene la declaración de terminales, no terminales, precedencia de operadores y reglas gramaticales del lenguaje GoLite. El parser permite validar estructuras como:

- Declaración de variables.
- Asignaciones.
- Operadores aritméticos.
- Operadores relacionales.
- Operadores lógicos.
- Sentencias `if` y `else`.
- Ciclos `for`.
- Instrucciones `break` y `continue`.
- Llamadas a funciones embebidas como `fmt.Println`.

Cuando se detecta un error sintáctico, el parser registra la información correspondiente para que pueda ser mostrada posteriormente al usuario.

7. Árbol de Sintaxis Abstracta

El proyecto utiliza una estructura basada en nodos para representar las instrucciones y expresiones reconocidas por el parser. Cada nodo representa una acción específica del lenguaje.

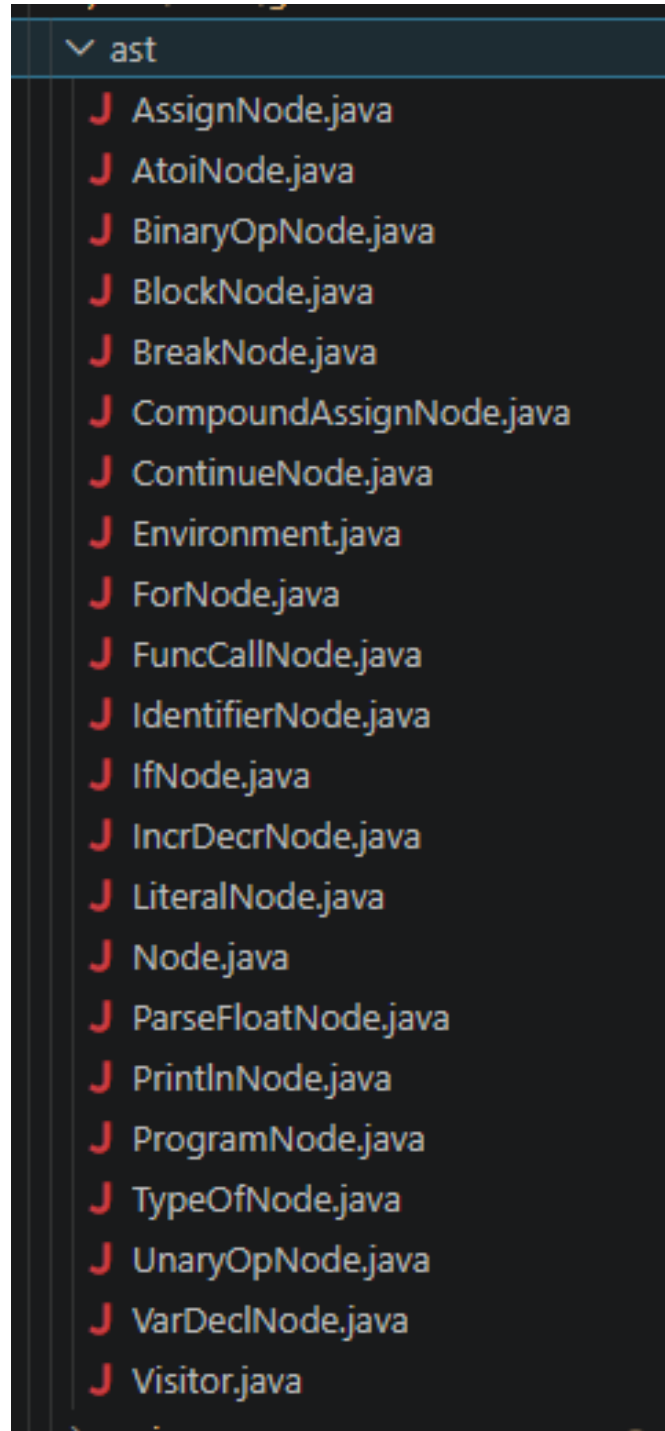


Figura 4: Clases utilizadas para representar nodos del AST

Entre los nodos implementados se encuentran:

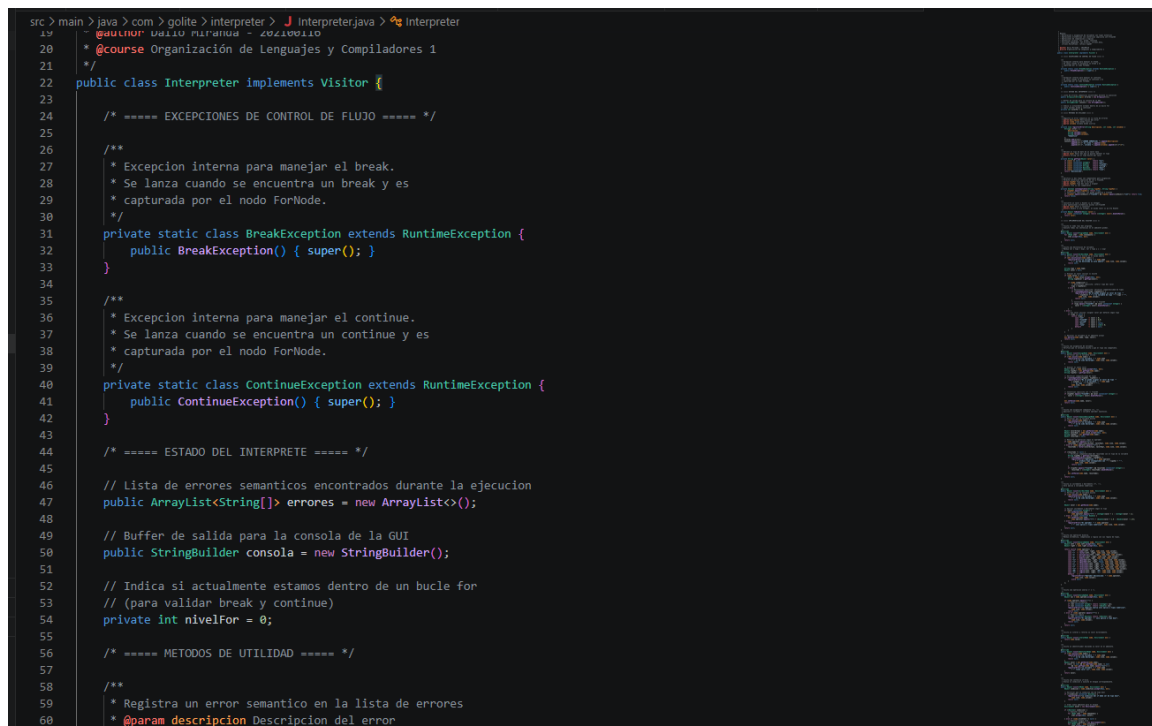
- **ProgramNode**: representa el programa completo.

- VarDeclNode: representa declaraciones de variables.
- AssignNode: representa asignaciones.
- BinaryOpNode: representa operaciones binarias.
- UnaryOpNode: representa operaciones unarias.
- IfNode: representa sentencias condicionales.
- ForNode: representa ciclos.
- PrintlnNode: representa llamadas a `fmt.Println`.

Esta organización facilita la interpretación del código, ya que cada instrucción puede ser procesada de manera independiente.

8. Módulo Intérprete

El módulo intérprete se encarga de recorrer los nodos generados por el parser y ejecutar las instrucciones correspondientes.



```

src > main > java > com > golite > interpreter > J interpreter.java > % interpreter
19  @author David Marín - 2020
20  * @course Organización de Lenguajes y Compiladores 1
21  */
22  public class Interpreter implements Visitor {
23
24      /* ===== EXCEPCIONES DE CONTROL DE FLUJO ===== */
25
26      /**
27       * Excepción interna para manejar el break.
28       * Se lanza cuando se encuentra un break y es
29       * capturada por el nodo ForNode.
30       */
31      private static class BreakException extends RuntimeException {
32          public BreakException() { super(); }
33      }
34
35      /**
36       * Excepción interna para manejar el continue.
37       * Se lanza cuando se encuentra un continue y es
38       * capturada por el nodo ForNode.
39       */
40      private static class ContinueException extends RuntimeException {
41          public ContinueException() { super(); }
42      }
43
44      /* ===== ESTADO DEL INTERPRETE ===== */
45
46      // lista de errores semanticos encontrados durante la ejecucion
47      public ArrayList<String[]> errores = new ArrayList<>();
48
49      // Buffer de salida para la consola de la GUI
50      public StringBuilder consola = new StringBuilder();
51
52      // Indica si actualmente estamos dentro de un bucle for
53      // (para validar break y continue)
54      private int nivelFor = 0;
55
56      /* ===== METODOS DE UTILIDAD ===== */
57
58      /**
59       * Registra un error semantico en la lista de errores
60       * @param descripcion Descripción del error

```

Figura 5: Clase Interpreter.java

El intérprete maneja:

- Declaración de variables.
- Validación de tipos.
- Operaciones aritméticas.

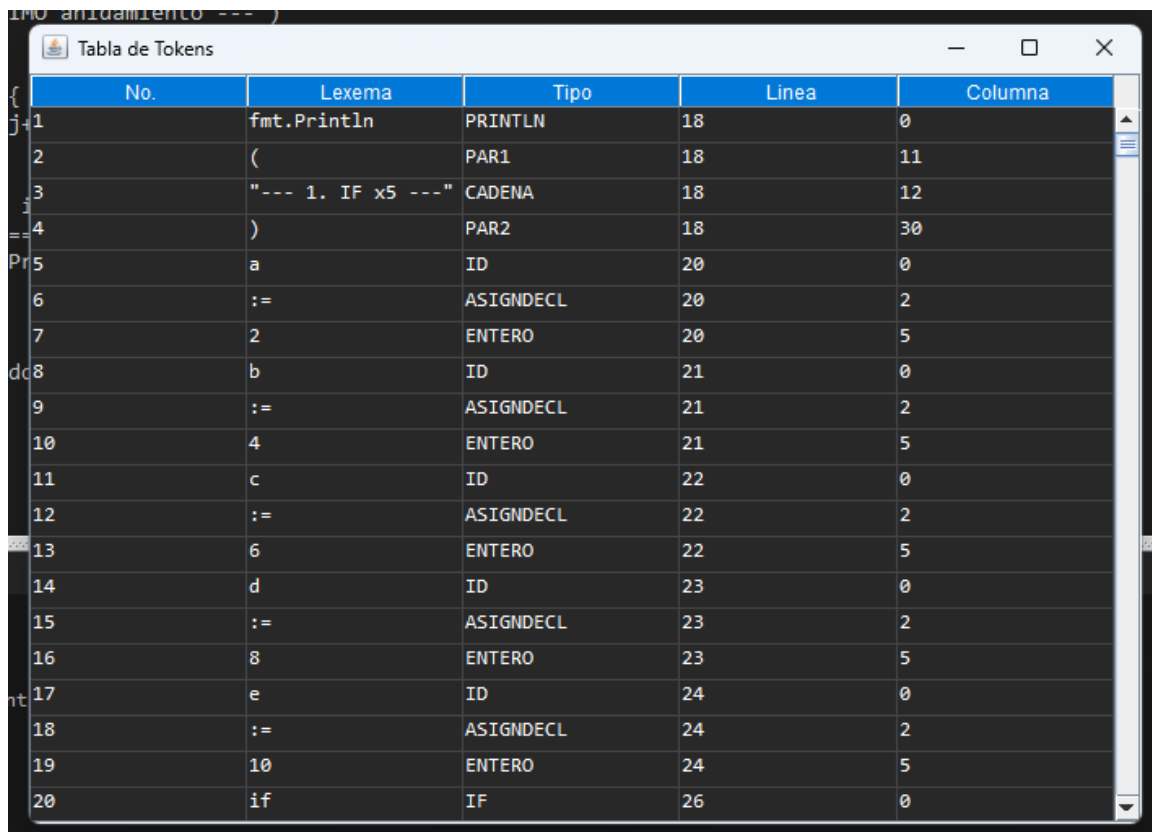
- Operaciones lógicas y relacionales.
- Ejecución de condicionales.
- Ejecución de ciclos.
- Manejo de `break` y `continue`.
- Registro de errores semánticos.
- Salida en consola.

9. Reportes Generados

El sistema genera reportes para facilitar la revisión del análisis realizado sobre el código fuente.

9.1. Reporte de Tokens

El reporte de tokens muestra cada lexema reconocido, su tipo, línea y columna.



No.	Lexema	Tipo	Linea	Columna
1	<code>fmt.Println</code>	PRINTLN	18	0
2	<code>(</code>	PAR1	18	11
3	<code>"--- 1. IF x5 ---"</code>	CADENA	18	12
4	<code>)</code>	PAR2	18	30
5	<code>a</code>	ID	20	0
6	<code>:=</code>	ASIGNDECL	20	2
7	<code>2</code>	ENTERO	20	5
8	<code>b</code>	ID	21	0
9	<code>:=</code>	ASIGNDECL	21	2
10	<code>4</code>	ENTERO	21	5
11	<code>c</code>	ID	22	0
12	<code>:=</code>	ASIGNDECL	22	2
13	<code>6</code>	ENTERO	22	5
14	<code>d</code>	ID	23	0
15	<code>:=</code>	ASIGNDECL	23	2
16	<code>8</code>	ENTERO	23	5
17	<code>e</code>	ID	24	0
18	<code>:=</code>	ASIGNDECL	24	2
19	<code>10</code>	ENTERO	24	5
20	<code>if</code>	IF	26	0

Figura 6: Reporte de tokens generado por el IDE

9.2. Reporte de Errores

Los errores léxicos, sintácticos y semánticos son registrados durante la ejecución. Estos errores son mostrados en la consola para que el usuario pueda corregir el programa.

10. Compilación del Proyecto

Para compilar el proyecto se utiliza Maven mediante el siguiente comando:

```
mvn clean package
```

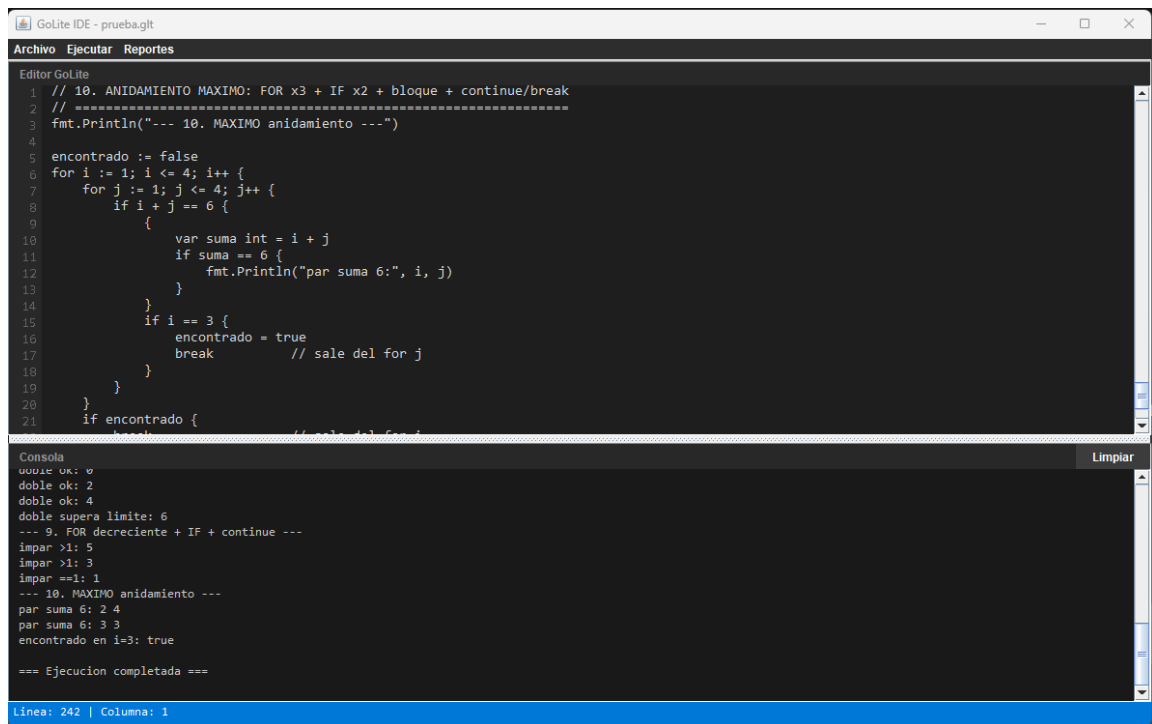
Al finalizar correctamente, Maven genera el archivo ejecutable del proyecto dentro de la carpeta `target`.

```
C:\Users\dalio\Documents\GoLite>mvn package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.golite:GoLite >-----
[INFO] Building GoLite 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- jflex:1.9.1:generate (default) @ GoLite ---
[INFO] Scanner.java is up to date.
[INFO]
[INFO] --- cup:11b-20160615-3:generate (default) @ GoLite ---
[INFO] parser.java is up to date.
[INFO]
[INFO] --- build-helper:3.6.0:add-source (add-cup-sources) @ GoLite ---
[INFO] Source directory: C:\Users\dalio\Documents\GoLite\target\generated-sources\cup added.
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ GoLite ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ GoLite ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ GoLite ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\dalio\Documents\GoLite\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ GoLite ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.5.4:test (default-test) @ GoLite ---
[INFO]
[INFO] --- jar:3.5.0:jar (default-jar) @ GoLite ---
[INFO]
[INFO] --- assembly:3.7.1:single (assemble-single) @ GoLite ---
[INFO] Building jar: C:\Users\dalio\Documents\GoLite\target\GoLite-1.0-SNAPSHOT-jar-with-dependencies.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.189 s
[INFO] Finished at: 2026-06-12T20:33:51-06:00
[INFO]
```

Figura 7: Compilación exitosa del proyecto con Maven

11. Ejecución del Sistema

Una vez compilado el proyecto, se puede ejecutar el archivo JAR generado. La interfaz permite escribir código GoLite y visualizar la salida en consola.



The screenshot shows the GoLite IDE interface. The top menu bar includes 'Archivo', 'Ejecutar', and 'Reportes'. The main editor window displays a Go program with the following code:

```
1 // 10. ANIDAMIENTO MAXIMO: FOR x3 + IF x2 + bloque + continue/break
2 // =====
3 fmt.Println("--- 10. MAXIMO anidamiento ---")
4
5 encontrado := false
6 for i := 1; i <= 4; i++ {
7     for j := 1; j <= 4; j++ {
8         if i + j == 6 {
9             {
10                 var suma int = i + j
11                 if suma == 6 {
12                     fmt.Println("par suma 6:", i, j)
13                 }
14             }
15             if i == 3 {
16                 encontrado = true
17                 break // sale del for j
18             }
19         }
20     }
21     if encontrado {
22         break // sale del for i
23     }
24 }
```

The bottom console window shows the execution output:

```
console ok: 0
doble ok: 2
doble ok: 4
doble supera limite: 6
--- 9. FOR decreciente + IF + continue ---
impar >1: 5
impar >1: 3
impar ==1: 1
--- 10. MAXIMO anidamiento ---
par suma 6: 2 4
par suma 6: 3 3
encontrado en i=3: true
=== Ejecucion completada ===
```

The status bar at the bottom indicates 'Lineas: 242 | Columna: 1'.

Figura 8: Ejecución correcta de un programa GoLite

12. Conclusiones

1. Se implementó un intérprete funcional para el lenguaje GoLite utilizando Java, JFlex y CUP.
2. El analizador léxico permite reconocer correctamente palabras reservadas, operadores, identificadores, literales y comentarios.
3. El analizador sintáctico valida la estructura de los programas mediante reglas gramaticales definidas en CUP.
4. La arquitectura basada en nodos facilita la ejecución del lenguaje y permite extender el proyecto en futuras fases.
5. La interfaz gráfica permite al usuario editar, ejecutar programas y visualizar reportes de forma sencilla.